

Analisi Approfondita dei Cambiamenti tra Sprint 0 e Sprint 1: Confronto Evolutivo e Zooming Architetturale nel Modello QAK

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 03, 2026

1. Introduzione e Obiettivi del Confronto (“Zooming Architettuale”)

Il presente documento analizza in modo rigoroso e puntuale tutte le differenze, le continuità logiche e le evoluzioni architetturelle intervenute nel passaggio dal modello QAK dello **Sprint 0** (`/sprint0/src/code.qak`) al modello prototipale dello **Sprint 1** (`/sprint1/prototype/codice_con_tutti_i_componenti.qak`).

La transizione tra i due Sprint rappresenta un perfetto esempio della metodologia di costruzione incrementale e di “**zooming architettuale**” adottata nel corso del Professor Natali:

- **Sprint 0 (Modello di Analisi / Dominio):** Ha lo scopo di definire l’architettura logica ad alto livello, stabilendo il lessico del dominio (Ubiquitous Language) e individuando le entità e le loro responsabilità principali. Nello Sprint 0 il comportamento del sistema non era ancora eseguibile né testabile; le azioni interne degli attori erano abbozzate tramite commenti testuali.
- **Sprint 1 (Prototipo Eseguibile del Core-Business):** Ha lo scopo di realizzare un primo prototipo interamente funzionante e testabile, focalizzato sul ciclo di gestione delle richieste di carico orchestrato dal `cargoservice`. Per permettere la verifica formale e l’esecuzione in isolamento senza dipendere dall’hardware fisico (robot, sensori, display reali) o da database di persistenza, si è ricorsi al pattern dei **Collaboratori Simulati (Mock)**.

2. Evoluzione della Topologia di Rete e dei Contesti QAK

Una delle modifiche più significative ed emblematiche del processo di affinamento software riguarda la strutturazione dei **Contesti (Context)** logici e di rete all’interno dell’infrastruttura QAK.

2.1. La scelta architetturelle dello Sprint 1: Il Contesto Unico (`ctxcargoservice`)

Nello Sprint 0 l’analisi del dominio aveva delineato 4 contesti elementari (uno per attore). Nel prototipo formale dello **Sprint 1** (`codice_con_tutti_i_componenti.qak`), si è operata una scelta ingegneristica precisa: **raggruppare tutti gli attori all’interno di un unico contesto** (`ctxcargoservice` sulla porta 8050).

ASPETTO	SPRINT 0 (<code>code.qak</code>)	SPRINT 1 (<code>codice_con_tutti_i_componenti.qak</code>)

			IS- TICA
Topologia Contesti	4 Contesti elementari (<code>ctxioport</code> , <code>ctxcargoservice</code> , <code>ctxhold</code> , <code>ctxrobot</code>)	1 Contesto Unico (<code>ctxcargoservice</code> sulla porta 8050)	Focalizzazione esclusiva sulla verifica algoritmica della Macchina a Stati del core-business, eliminando complessità di rete premature.
Comunicazione	Ipotizzata via TCP tra 4 nodi separati	Messaggi locali e reattivi in memoria (stessa JVM)	Garantisce testabilità im-

			me- di- ata, as- senza di latenza/ over- head TCP e col- laudo pu- ra- mente fun- zionale in iso- la- mento.
Evoluzione Futura (Sprint 2)	N/A	Architettura Multi- Contesto distribuita prepianificata in utils/prototype/	I prog- etti sep- a- rati in con- testi fisici , ctxcustome , ctxdevices ecc.) cos- ti- tu- is- cono la base evo-

--	--	--

luta
per
il
prossimo
Sprint.

2.2. Motivazione Ingegneristica del Contesto Unico per lo Sprint 1

In un processo incrementale ben governato, ogni Sprint deve isolare e risolvere un singolo sottoproblema (**Separation of Concerns**). L'obiettivo cardine dello Sprint 1 è verificare che la logica di orchestrazione, le transizioni di stato del `cargoservice` e i protocolli di interazione con i mock siano esenti da deadlock o errori concettuali. Eseguire l'intero sistema in un **unico contesto logico** (`ctxcargoservice`) permette di:

1. Effettuare il collaudo funzionale immediato e il debugging deterministico della Macchina a Stati Finiti.
2. Evitare che eventuali problematiche di configurazione di rete, firewall o serializzazione su socket TCP mascherino errori logici dell'algoritmo di carico.

2.3. La Base per lo Sprint 2: I Progetti Multi-Contesto in `utils/prototype/`

L'evoluzione architetturale successiva — ovvero la distribuzione su nodi fisici computazionali separati (`ctxcustomer` per l'interfaccia cliente, `ctxdevices` per stiva e sensori di campo, `ctxrobot` per il movimentatore) — è **già stata progettata e pre-disposta all'interno della directory `utils/prototype/`**. Questa versione distribuita rappresenta lo stadio di maturazione immediatamente successivo al prototipo odierno e costituirà la **base di partenza ideale per lo Sprint 2**, quando i collaboratori simulati verranno progressivamente sostituiti con le GUI Web reali e i dispositivi fisici/virtuali connessi via TCP/IP.

3. Lessico e Vocabolario delle Interazioni: Continuità e Arricchimento

3.1. Continuità Assoluta nel Protocollo di Core-Business

Il contratto di comunicazione tra il Cliente (IOPort) e l'Orchestratore (CargoService) è rimasto **impeccabilmente identico**:

```
// SPRINT 0 e SPRINT 1 (Identici al 100%)
Request load_request      : loadRequest(none)
Reply   load_accepted     : loadAccepted(SLOTID) for load_request
Reply   load_retrylater   : loadRetryLater(none) for load_request
Reply   load_refused      : loadRefused(none) for load_request
```

Questo dimostra l'efficacia dell'Analisi dei Requisiti condotta nello Sprint 0: le firme e la semantica dei messaggi di business non hanno subito alcuna alterazione.

3.2. Arricchimento del Vocabolario di Interazione (Nuovi Messaggi)

Per trasformare le azioni descritte a parole in comunicazioni reali, nello Sprint 1 sono stati introdotti nuovi messaggi formali:

1. Interazione con il Magazzino (Hold):

- **Sprint 0:** Nessun messaggio formale di richiesta slot.
- **Sprint 1:** Introdotte `Request get_slot : getSlot(none)`, `Reply slot_reserved : slotReserved(SLOTID)`, `Reply hold_full : holdFull(none)` e `Dispatch free_slot : freeSlot(SLOTID)`.
- **Motivazione:** L'adozione del pattern **Request/Reply** verso l'attore `hold` è fondamentale perché la prenotazione dello slot richiede una **risposta certa e asincrona**: il `cargoservice` non può rispondere al cliente se prima non ottiene dalla stiva l'ID dello slot libero o la notifica di stiva piena. Il `Dispatch free_slot` serve invece per liberare lo slot in caso di timeout del deposito container.

2. Interazione con Sensori e Attuatori (Sonar, LED, Robot, Marker):

- **Sprint 0:** Assenti nel codice QAK.
- **Sprint 1:**
 - ▶ `Event sonardata : distance(D)`: Emesso dal sonar per comunicare variazioni fisiche di distanza nell'area di carico.
 - ▶ `Dispatch set_service_status : setServiceStatus(STATUS)`: Inviato dal sonar per forzare il sistema nello stato "working" o "outofservice" in caso di guasto prolungato.
 - ▶ `Dispatch led_ctrl : ledCmd(CMD)`: Comando unidirezionale ("on", "off", "blink") verso il LED.
 - ▶ `Request robot_move : robotMove(TARGET)` / `Reply robot_done`: Pattern Request/Reply per comandare e attendere il completamento delle movimentazioni del robot.
 - ▶ `Request mark_container : markContainer(none)` / `Reply marking_done`: Pattern Request/Reply per il processo di etichettatura.

4. Evoluzione Comportamentale e Macchina a Stati del `cargoservice`

L'attore `cargoservice` ha subito la trasformazione più profonda, passando da un modello puramente descrittivo a una vera e propria **Macchina a Stati Finiti (FSM) asincrona e reattiva**.

4.1. Rinominazione e Split dello Stato di Attesa: da `work` a `disengaged` ed `engaged`

- **Nello Sprint 0:** Il `cargoservice` possedeva un unico stato di attesa generico chiamato `work`.
- **Nello Sprint 1:** Lo stato `work` è stato eliminato ed è stato sostituito/sdoppiato in due stati di dominio espliciti: `disengaged` ed `engaged`.
- **Perché questo cambio è cruciale?** La modifica allinea perfettamente il modello QAK alla terminologia ufficiale richiesta dal committente nel testo dei requisiti: “altrimenti considera il sistema come `engaged` [...] mentre il sistema è `engaged` il LED deve lampeggiare [...] altrimenti il sistema diventa `disengaged`”. Avere due stati distinti rende immediata e formale la distinzione tra sistema libero e sistema occupato in un ciclo di carico.

4.2. Guardia Logica sulle Precondizioni in `handle_load_request`

Nello Sprint 0 le condizioni per rifiutare o rimandare una richiesta erano commenti. Nello Sprint 1 sono diventate una guardia di controllo booleana (in stile Macchina di Moore):

```
// SPRINT 1: Controllo rigoroso delle precondizioni di accettazione
if [# CargoState = "engaged" || !ServiceWorking || IOPortOccupied #] {
    replyTo load_request with load_retrylater : loadRetryLater(none)
} else {
    request hold -m get_slot : getSlot(none)
}
```

4.3. Srotolamento Asincrono (“Unrolling”) del Flusso di Carico

Nello Sprint 0, all’interno dello stato `handle_load_request`, l’intero processo di carico era riassunto in un blocco di commenti:

```
/* SPRINT 0 (solo commento):
 * Dopo accettazione: attende container entro tempo prefissato;
 * usa cargorobot per spostare da IOPort a slot5; attende marcatura;
 * usa cargorobot per spostare da slot5 a slot riservato.
 */
```

Nello Sprint 1, per rispettare il paradigma di comunicazione asincrona a messaggi di QAK senza bloccare il thread dell’attore, quel singolo commento è stato “srotolato” in **7 stati operativi sequenziali e reattivi**:

1. `wait_for_slot` : Attende la risposta (`slot_reserved` o `hold_full`) dall’attore `hold`.
2. `accept_request` / `refuse_request` : Invia la Reply finale al cliente (`load_accepted(ID)` con attivazione LED lampeggiante o `load_refused`).
3. `handle_sonar` : Ascolta l’evento `sonardata` per aggiornare la variabile booleana `IOPortOccupied` e rilevare la presenza fisica del container nell’area di carico.

4. `do_robot_job` : Invia la Request `robot_move(slot5)` al robot non appena il container viene rilevato.
5. `mark_container` : Al ricevimento di `robot_done` , invia la Request `mark_container` al marcatore.
6. `move_to_reserved_slot` : Al ricevimento di `marking_done` , comanda al robot il movimento finale `robot_move(slotID)` .
7. `finish_job` : Al completamento, spegne il LED, riporta `CargoState = "disengaged"` e torna in ascolto.

4.4. Resilienza, Timeout e Gestione Guasti

Nello Sprint 1 è stata introdotta la gestione esplicita dei casi limite e di errore:

- **Timeout Deposito Container:** Nello stato `engaged` è presente la transizione `whenTime 30000 → handle_deposit_timeout` . Se il cliente non deposita il container entro 30 secondi, il `cargoservice` annulla la prenotazione inviando un `Dispatch free_slot` all’hold, spegne il LED e torna `disengaged` .
- **Guasto Sonar (Out of Service):** La transizione `whenMsg set_service_status → update_service` intercetta le notifiche del sonar e imposta `ServiceWorking = false` , garantendo che successive `load_request` ricevano immediatamente `load_retrylater` .

5. Tabella Sintetica di Confronto Stato per Stato (`cargoservice`)

STATO / ELEMENTO	SPRINT 0 (<code>code.qak</code>)	SPRINT 1 (<code>codice_con_tut</code>)	EVOLUZIONE
<code>s0</code>	Presente	Presente	Stato di boot e in-izial-iz-zazione vari-abili.

work	Presente (Stato attesa)	Eliminato	Sos- ti- tu- ito dagli stati di do- minio ed disengaged . engaged
disengaged	Assente	Nuovo (da work)	Sis- tema libero, LED spento, in at- tesa di richi- este di carico.
engaged	Assente	Nuovo (da work)	Sis- tema im- peg- nato in una pro- ce- dura di carico, LED lam- peg- giante.
handle_load_request	Presente (solo commenti)	Presente (logica es- eguibile)	Im- ple-

			menta la guardia sulle vari- abili in- terne e in- ter- roga la . hold
wait_for_slot	Assente	Nuovo	Stato di at- tesa as- in- crona della Re- ply dalla . hold
accept_request / refuse_request	Assenti (inline)	Nuovi	Smis- ta- mento for- male delle risposte verso il Cus- tomer.
handle_sonar / update_service	Assenti	Nuovi	Ges- tione degli eventi di

			pre- senza con- tainer e guasto sonar.
do_robot_job	Assente (commento)	Nuovo	Richi- esta movi- mento verso slot5
mark_container	Assente (commento)	Nuovo	Richi- esta etichet- tatura al markerdevice
move_to_reserved_slot	Assente (commento)	Nuovo	Richi- esta movi- mento verso lo slot fi- nale ris- er- vato.
finish_job	Assente	Nuovo	Chiusura transazione, speg- n- i- mento LED, ri- torno a disengaged

<code>handle_deposit_timeout</code>	Assente	Nuovo	Ges- tione sca- denza 30s con ri- las- cio slot) <code>. free_slot</code>
-------------------------------------	---------	-------	--

6. Impatto sulla Testabilità e Valore del Prototipo

Il passaggio dal modello dello Sprint 0 a quello dello Sprint 1 rappresenta un salto qualitativo decisivo nell'ingegneria del software applicata al progetto:

1. **Dalla Specifica al Software Eseguibile:** Mentre `code.qak` nello Sprint 0 era un documento formale di specifica non eseguibile, il codice dello Sprint 1 compila in attori Kotlin pienamente operativi all'interno del contesto di collaudo.
2. **Isolamento e Collaudo (Mocking in Contesto Unico):** L'utilizzo degli attori mock nel contesto unico `ctxcargoservice` permette di validare l'orchestratore senza interferenze esterne. È possibile simulare scenari complessi (come guasti improvvisi o stiva piena) con determinismo e velocità di esecuzione ottimali.
3. **Automazione dei Test Plan:** Grazie a questa architettura, i test di accettazione T1, T2 e T3 definiti nello Sprint 0 sono stati automatizzati in suite JUnit (classe `TestCargoServiceCore.kt`). Tali test agiscono come client che inviano le stringhe Prolog di richiesta (`msg(load_request, ...)`) al server QAK e verificano le risposte del sistema in modo certo e ripetibile.